Project : MicroGoal

Team Members :

G. Samiksha

[Register No. : 212402985

N.M ID : B006314EC615B7694BC4140B75627E10]

R. Thamizharasi

[Register No.: 212403005

N. M ID:02E062841BFFA9DB3DA4ACF5237D2E1E]

P. Raveena

[Register No. : 212402976

N. M ID: D686CE71DEF13609C524FB4048CE4D]

Frontend Development with React.js

Project Documentation format

1. **Introduction**
    - **Project Title**: MicroGoal
    - **Team Members**:
    - G. Samiksha [212402985]
    - R. Thamizharasi [212403005]
    - P. Raveena[212402976]
2. **Project Overview**
    - **Purpose**: Briefly describe the purpose and goals of the project.
    - **Features**: Highlight the key features and functionalities of the frontend.
3. **Architecture**
    - **Component Structure**: Outline the structure of major React components and how they interact.
    - **State Management**: Describe the state management approach used (e.g., Context API, Redux).
    - **Routing**: Explain the routing structure if using react-router or another routing library.
4. **Setup Instructions**
    - **Prerequisites**: List software dependencies (e.g., Node.js).
    - **Installation**: Provide a step-by-step guide to clone the repository, install dependencies, and configure environment variables.
5. **Folder Structure**
    - **Client**: Describe the organization of the React application, including folders like components, pages, assets, etc.
    - **Utilities**: Explain any helper functions, utility classes, or custom hooks used in the project.
6. **Running the Application**
    - Provide commands to start the frontend server locally.
        - **Frontend**: npm start in the client directory.
7. **Component Documentation**
    - **Key Components**: Document major components, their purpose, and any props they receive.
    - **Reusable Components**: Detail any reusable components and their configurations.
8. **State Management**
    - **Global State**: Describe global state management and how state flows across the application.
    - **Local State**: Explain the handling of local states within components.
9. **User Interface**
    - Provide screenshots or GIFs showcasing different UI features, such as pages, forms, or interactions.
10. **Styling**

- **CSS Frameworks/Libraries**: Describe any CSS frameworks, libraries, or pre-processors (e.g., Sass, Styled-Components) used.
- **Theming**: Explain if theming or custom design systems are implemented.

11. **Testing**

- **Testing Strategy**: Describe the testing approach for components, including unit, integration, and end-to-end testing (e.g., using Jest, React Testing Library).
- **Code Coverage**: Explain any tools or techniques used for ensuring adequate test coverage.

12. **Screenshots or Demo**

- Provide screenshots or a link to a demo showcasing the application's features and design.

13. **Known Issues**

- Document any known bugs or issues that users or developers should be aware of.

14. **Future Enhancements**

- Outline potential future features or improvements, such as new components, animations, or enhanced styling.

MicroGoals – Secure Goal Management Backend API

Project Overview

MicroGoals is a secure backend RESTful API developed using Node.js, Express.js, and MongoDB. It enables users to register, log in, and manage their personal goals through CRUD operations. The system uses JWT authentication and middleware to ensure data security and user-specific access control.

The system includes:

- User Registration & Login
- JWT-based Authentication
- CRUD Operations for Goals
- Protected Routes
- Centralized Error Handling

The main objective is to provide a secure and scalable backend system for digital goal management.

Project Instruction

- The project follows REST API principles.
- Authentication is handled using JWT.
- MongoDB is used as the database.
- APIs are tested using Thunder Client.

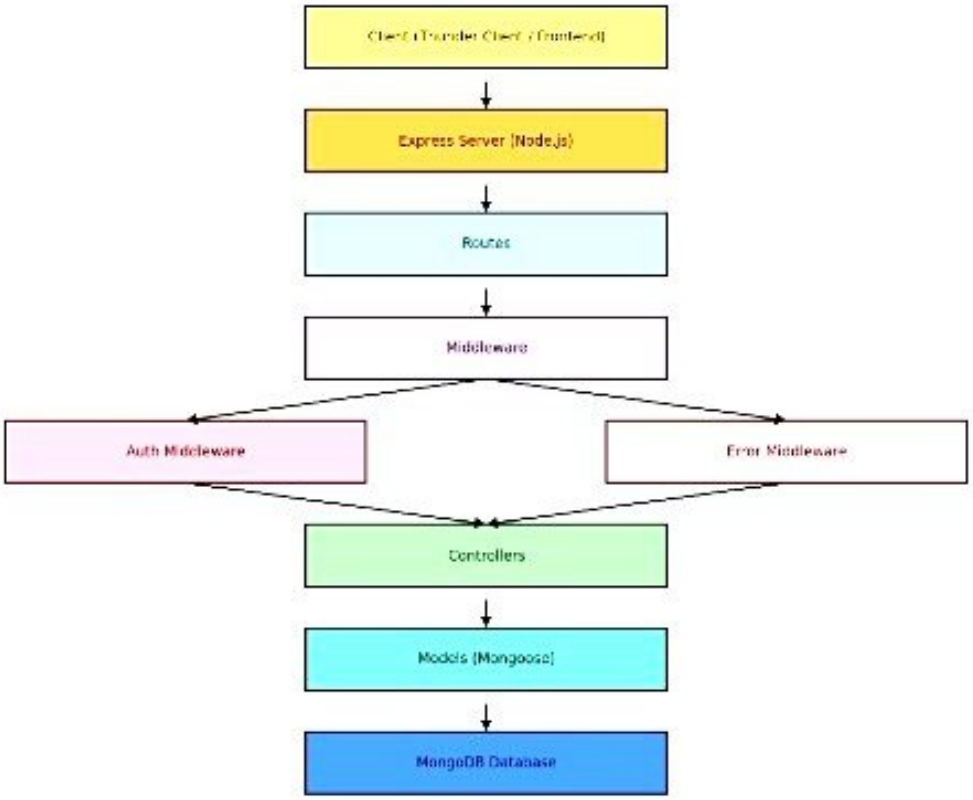
- The project follows MVC architecture.

Pre-requisites

Before running the project, the following tools are required:

- Node.js (v18 or higher)
- Express.js
- MongoDB (Local / Atlas)
- VS Code
- Thunder Client (Extension)
- Basic knowledge of JavaScript & REST APIs

Project Architecture Image (Text Representation)



PROJECT ARCHITECTURE

TECHNICAL ARCHITECTURE

- *Runtime: Node.js*
- *Framework: Express.js*
- *Database: MongoDB*

- *Authentication: JWT*
- *Architecture Pattern: MVC*

ER DIAGRAM

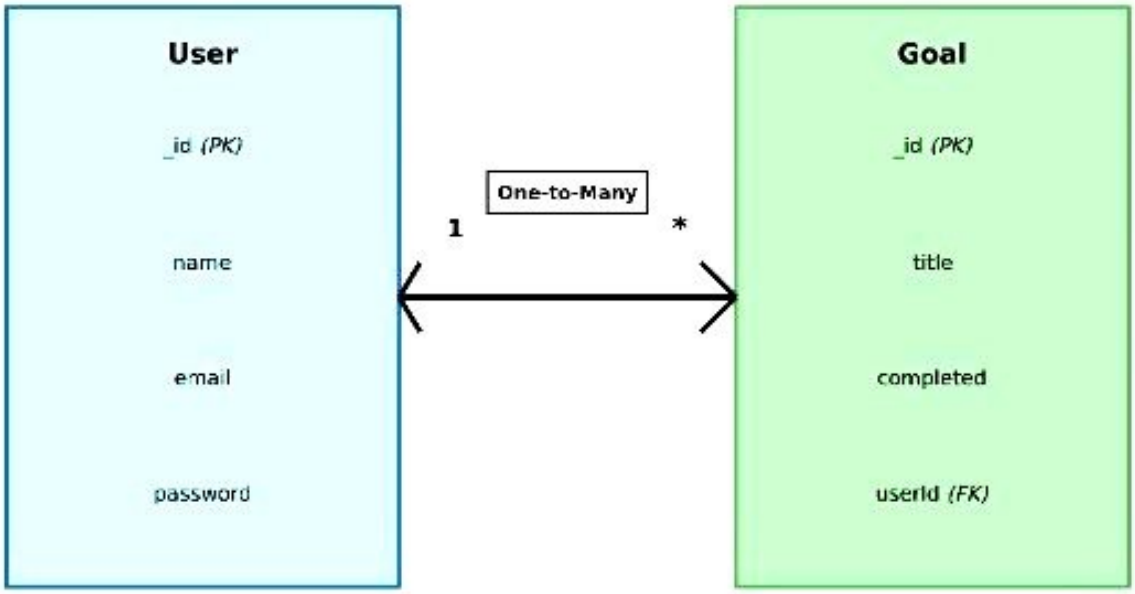
Entities:

1. *User*
    - *_id*
    - *name*
    - *email*
    - *password*
2. *Goal*
    - *_id*
    - *title*
    - *completed*
    - *userId (Reference to User)*

Relationship:

One User → Many Goals

User entities link to multiple Goal entities via foreign key



FEATURES

- *User Registration*
- *Secure Login with JWT*
- *Create Goal*
- *Update Goal*

- Delete Goal
- View User-specific Goals
- Protected Routes
- Centralized Error Handling

ROLES AND RESPONSIBILITIES

Role	Responsibility
User	Register, Login, Manage Goals
Developer	API Development & Testing

USERS FLOW

1. User registers
2. User logs in
3. JWT token generated
4. User sends token in header

5. *Access protected routes*

6. *Perform CRUD operations*

PROJECT SETUP AND CONFIGURATION

- **Folder Structure**

MicroGoals/

```
├── middleware/
│   ├── authMiddleware.js
│   └── errorMiddleware.js
├── Schema/
├── User.js
├── GoalSchema.js
├── models/
│   ├── userModel.js
│   └── goalModel.js
│
├── routes/
│   ├── authRoutes.js
│   └── goalRoutes.js
│
├── .env
├── Index.js
└── package.json
```

MicroProjectApi
Backend
Middlewares
authMiddleware.js
errormiddleware.js
Model
GoalModel.js
UserModel.js
node_modules
Router
AuthRouter.js
GoalRouter.js
Schema
GoalSchema.js
User.js
utills
genratetoken.js
validation
userValidation.js
.env
.gitignore
Index.js
package-lock.json
package.json
Frontend
Portfolio
PracticeBackend
Project_simon says
OUTLINE

- ***Project Setup***

1. *npm init -y*

2. *Install dependencies:*

npm install express mongoose jsonwebtoken bcryptjs dotenv

3. *Start server:*

node server.js

BACKEND DEVELOPMENT

- *Created Express Server*

An Express server was set up to handle API requests, connect routes, and enable JSON parsing for backend operations.

JS Index.js ×

```javascript
require("dotenv").config();
const express = require("express");
const app = express();
const mongoose = require("mongoose");
const cors = require("cors");
app.use(cors());
const errormiddleware = require("./Middlewares/errormiddleware");
app.use(express.urlencoded({ extended: true }));
const bodyParser = require("body-parser");
app.use(express.json());
const PORT = process.env.PORT;
const Mongo_Url = process.env.MONGO_URL;
const authRoute = require("./Router/AuthRouter");
const GoalRouter = require("./Router/GoalRouter");

//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));

//route
app.use("/", authRoute);
app.use("/", GoalRouter);

app.use(errormiddleware);
app.listen(PORT, () => {
  console.log("Server running on port 8000");
});
```

- *Connected MongoDB*

For Connection with MongoDB:-

```
//mongoDB connection
mongoose
  .connect(Mongo_Url)
  .then(() => console.log("MongoDB Connected"))
  .catch((err) => console.log(err));
```

- *Implemented Authentication APIs*

User registration and login APIs were created with password hashing and JWT token generation for secure authentication.

```
index.js        authRoutes.js ×
const express = require('express');
const router = express.Router();
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const uservalidation = require('../validation/uservalidation');
const userModel = require('../Model/UserModel');
const generateToken = require('../utils/generateToken');
router.post('/signup', async (req, res, next) => {
  try {
    const { error } = uservalidation.validate(req.body);
    if (error) {
      return res.status(400).json({
        success: false,
        message: "enter the data correctly",
      });
    }
    const { name, email, password } = req.body;
    const exist = await userModel.findOne({ email });
    if (exist) {
      return res.status(400).json({ message: "User already exists" });
    }
    const hashpassword = await bcrypt.hash(password, 10);
    const user = new userModel({
      name: name,
      email: email,
      password: hashpassword,
    });
    await user.save();
    const token = generateToken(user._id);
    res.status(200).json({
      success: true,
      message: "user is created",
      token: token,
      data: user,
    });
  } catch (err) {
    return next(err);
  }
});

router.post('/login', async (req, res, next) => {
  try {
    const { email, password } = req.body;
    const user = await userModel.findOne({ email });

    if (!user) {
      const err = new Error("user not found");
      err.status = 400;
      return next(err);
    }

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) {
      const err = new Error("Wrong Password");
      err.status = 400;
      return next(err);
    }
    const token = generateToken(user._id);
    res.status(200).json({
      success: true,
      message: "login successfully",
      token: token,
      data: user,
    });
  } catch (error) {
    return next(error);
  }
});
```

- *Implemented Goal CRUD APIs*

Goal CRUD APIs were implemented to allow authenticated users to perform full Create, Read, Update, and Delete operations on their personal goals. These APIs follow RESTful architecture principles and are designed to ensure secure and structured data management.

The Create API enables users to add new goals to the database. Each goal is associated with a

specific user using a userId reference, ensuring user-specific data isolation.

The Read API allows authenticated users to retrieve only their own goals from the database. Data filtering is applied using the authenticated user's ID to prevent unauthorized data access.

The Update API enables users to modify existing goals. Proper validation checks ensure that only the owner of the goal can update it.

The Delete API allows users to permanently remove their goals from the system. Secure route protection ensures that users cannot delete goals belonging to others.

All CRUD routes are protected using Authentication Middleware, which verifies the JWT token before granting access. If the token is missing or invalid, the system returns a 401 Unauthorized error.

Additionally, Error Middleware is implemented to handle runtime and validation errors centrally. It captures exceptions from CRUD operations and returns consistent JSON responses with appropriate HTTP status codes such as 200, 201, 400, 401, and 404.

This structured implementation ensures secure data handling, proper authorization, and reliable API behavior across all goal management operations.

```
const express = require('express');

const router = express.Router();

const goalModel = require('../Models/GoalModel');

const authMiddleware = require('../Middlewares/authMiddleware');


router.post('/create', authMiddleware, async (req, res, next) => {
```

media.events = router;

- *Added Auth Middleware*
- *Auth middleware was implemented to verify JWT tokens and secure protected routes from unauthorized access.*

JS Index.js JS authMiddleware.js ×

```javascript
const jwt = require("jsonwebtoken");
require("dotenv").config();

const authmiddleware = (req, res, next) => {
  try {
    const header = req.headers.authorization;

    if (!header) {
      return res.status(401).json({ message: "No token" });
    }

    const token = header.split(" ")[1];

    const decoded = jwt.verify(token, process.env.SECRET_KEY);

    req.user = decoded.userId;

    next();
  } catch (error) {
    return res.status(401).json({ message: "Invalid token" });
  }
};
module.exports = authmiddleware;
```

- *Added Error Middleware*

An error middleware was added to handle application errors centrally and return proper status codes and messages.

JS errormiddleware.js ×

```javascript
const errormiddleware = (err, req, res, next) => {
  console.log(err);
  const status = err.status.code || 500;
  const message = err.message || "server error";
  res.status(status).json({
    success: false,
    message: message,
  });
};

module.exports = errormiddleware;
```

DATABASE DEVELOPMENT

CONFIGURE MongoDB

- *Use MongoDB Atlas or Local MongoDB*

- *Store connection string in .env*

CREATE DATABASE CONNECTION

Index.js file:

- *Use mongoose.connect()*

/

- Handle connection errors

CREATE SCHEMA AND MODELS

User Schema

The User Schema defines the user data structure with name, email (unique), and hashed password fields.

```
JS User.js
const mongoose = require("mongoose");
const Schema = mongoose.Schema;
const userSchema = new Schema({
  name: {
    type: String,
    required: true,
  },
  email: {
    type: String,
    unique: true,
  },
  password: {
    type: String,
    required: true,
  },
});
module.exports=userSchema;
```

Goal Schema (with userId reference)

The Goal Schema stores goal details and includes a userId reference to associate each goal with a specific user.

JS GoalSchema.js ×

```javascript
const mongoose = require("mongoose");
const Schema = mongoose.Schema;

const goalSchema = new Schema({
  title: {
    type: String,
    required: true,
    trim: true,
  },

  completed: {
    type: Boolean,
    default: false,
  },

  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
    required: true,
  },
});

module.exports=goalSchema;
```

PROJECT EXECUTION

Project Execution Steps

1. Start MongoDB

2. Run server

3. Test APIs in Thunder Client

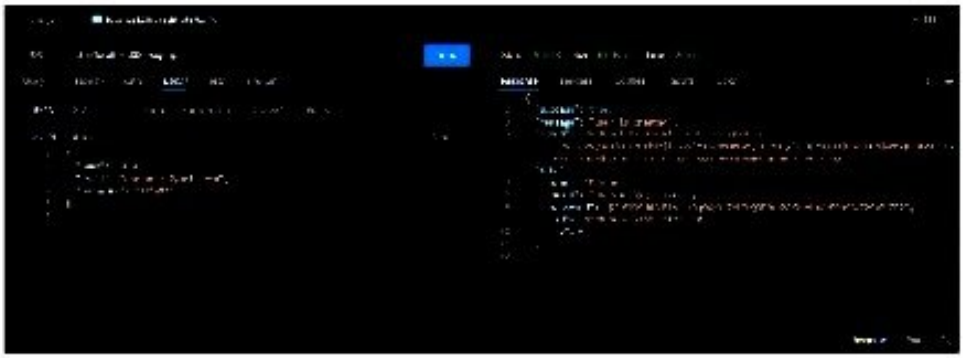
4. Register User

5. Login and get JWT

6. Perform CRUD operations

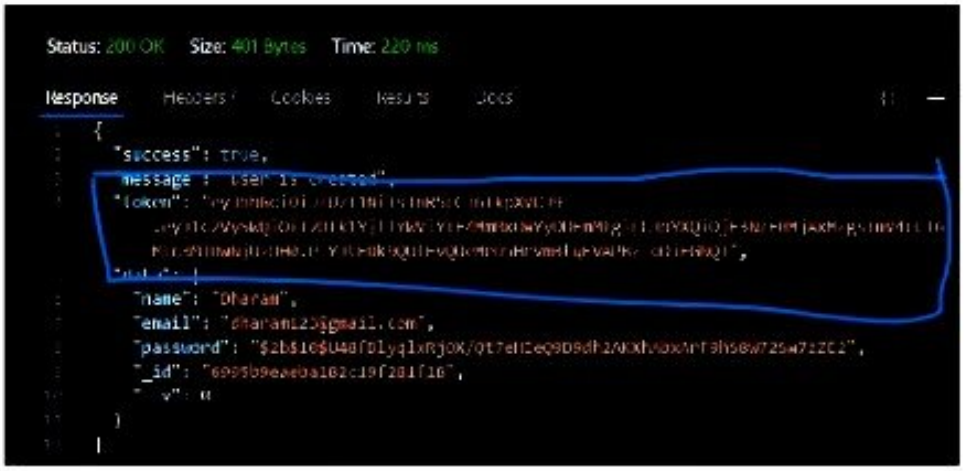
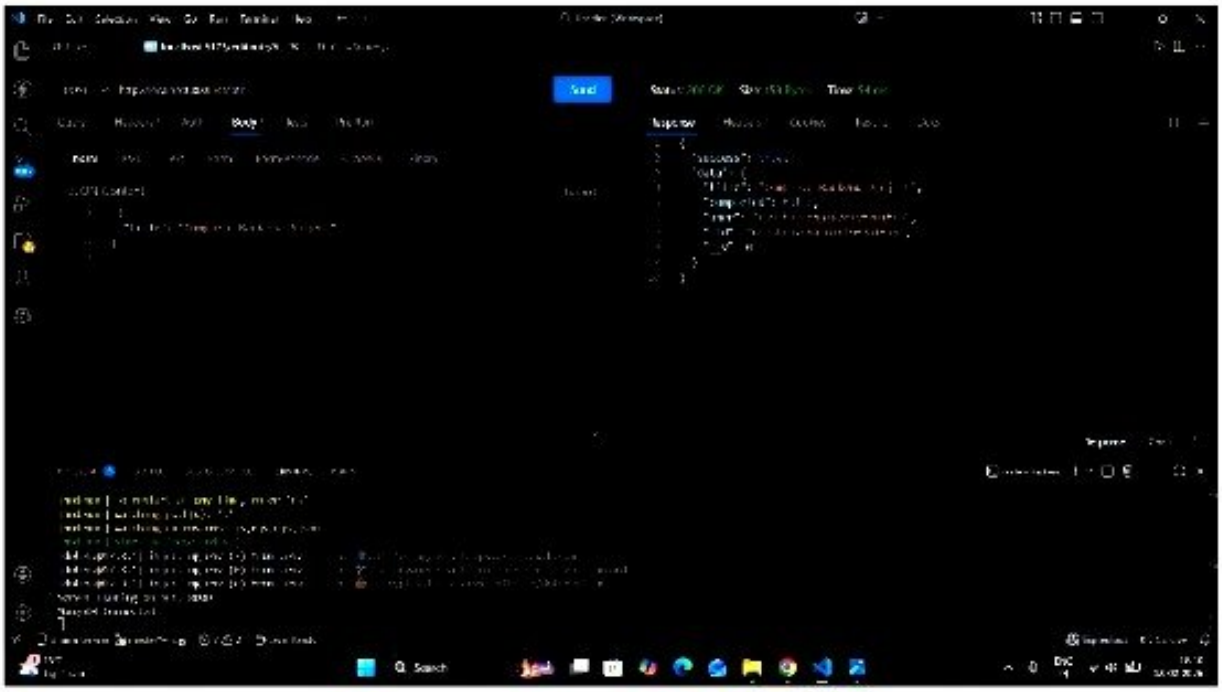
Project Output Screenshots

- Successful User Registration(Signup/Login)
- In this Api call the user register or login and token was genrated



- JWT Token Response

After successful login user the api generate the token and these token was sent with every request to every api



- *Goal Created Successfully*
- *After the login and token is generated the user give post request to api for creating there goals and goal was save in MongoDB which was the database*